Runtime Variant Selection in LLVM's GPU Offload Stack: Metadata, Measurement, and Mechanism

Akash
IIT Patna, CERN GSoC Alumnus, vLLM Contributor

EuroLLVM Developers' Meeting, Dublin 2026
April 2026

Abstract

MLIR can compile a single `gpu.module` to multiple GPU vendors (NVIDIA, AMD, Intel) since August 2025. The `OffloadBinary` container can carry N device images. But at runtime, LLVM picks the *first compatible image and stops*—there is no standard way to select the *best* one.

We present three contributions: (1) a standard metadata vocabulary for `OffloadBinary` describing image requirements and quality (five new keys, backward-compatible); (2) the first published flame graph decomposing the LLVM GPU dispatch stack latency from binary parse through kernel launch (measured on GTX 1650); (3) a design sketch for `#gpu.runtime_select`, an MLIR attribute enabling best-compatible selection at runtime.

The concrete contributions are (1) and (2). The design sketch (3) motivates them and shows where the measured metadata will be consumed.

1 Introduction

LLVM's offload compiler pipeline has reached a critical inflection point. Since August 2025 (PR #148286, merged), MLIR's `gpu-module-to-binary` pass supports compiling a single source to three vendor targets: NVIDIA (`#nvvm.target`), AMD (`#rocdl.target`), and Intel (`#xevm.target`). The `OffloadBinary` fat-binary format (D122069, 2022) can carry all three device images in a single container.

Yet the runtime has not caught up. When `parseOffloadBinary` iterates through images at load time (PR #186088), it selects the *first* one whose triple and architecture match the current device, then stops. There is no mechanism to measure what each image requires, no way to express image quality tiers, and no policy to select among multiple compatible options.

This gap is no longer theoretical. Downstream users work around it today:

- **HEP-CCE (CERN CMS):** Maintains 80 separate build configurations to target heterogeneous GPU clusters (NVIDIA A100/V100 + AMD MI250X + CPU fallback). A single fat binary with runtime selection eliminates this combinatorial explosion.
- **vLLM:** Maintains separate NVIDIA and AMD codepaths with different kernel implementations. Runtime variant selection would allow a single binary serving both.
- **Cloud GPU containers:** AWS p4/p5 instances and Google Cloud A3 have unknown GPU models at container build time. Runtime selection is the only practical option.

We propose a three-layer solution: a standard metadata vocabulary (Contribution 1) that describes what each image requires and prefers; a dispatch flame graph (Contribution 2) measuring the cost this metadata must fit into; and a compiler attribute design (Contribution 3) showing how the metadata enables runtime selection.

2 Background

2.1 The Multi-Vendor Compilation Gap

The pipeline from `gpu.module` to executable binary is now complete:

```
gpu.module @kernels {
  gpu.func @kernel(...) {
    ...
  }
}

gpu.module_to_binary @kernels to_targets(
  #nvvm.target<chip = "sm_75">,
  #rocdl.target<chip = "gfx90a">,
  #xevm.target<device = "pvc">
) -> gpu.binary @kernels
```

The `gpu-module-to-binary` pass compiles the module three times and produces:

$$\texttt{gpu.binary} = [\text{cubin}_{75}, \text{hsaco}_{90a}, \text{spirv}_{pvc}]$$

This binary is wrapped in `OffloadBinary` at clang-linker-wrapper time. The format is a flexible key-value container with a string map per image:

Key	Type	Since
`triple`	string	D122069 (2022)
`arch`	string	D122069 (2022)

The runtime consumer hook exists: `isMetadataCompatible()` (PR #185663, merged March 2026) filters images in the `parseOffloadBinary` loop. **The consumer exists. The vocabulary does not.**

2.2 The Selection Policy Vacuum

`#gpu.select_object` is the sole implementation of `OffloadingLLVMTranslationAttrInterface` in LLVM mainline. It resolves the binary choice at compile time by index or static target match:

```
gpu.binary @kernels <#gpu.select_object<0>> [...]
```

This commits to a single image at LLVM-IR translation time. RFC #88170 (GPU Dialect cleanup) explicitly separates the container ("gpu.binary will hold N objects") from the policy slot ("who decides which one runs?"). The policy slot is vacant.

2.3 Measurement Gap

Nobody has published a per-layer latency breakdown of the full LLVM GPU dispatch path. Endpoints are known: TaxBreak [1] measures null-kernel dispatch on H100 via CUDA driver API directly: 4.707 μs average (p50: 4.578 μs). PyTorch eager dispatch is 5–10 μs per kernel [2]. But the interior—how time distributes across `OffloadBinary` parse, `olCreateProgram`, `cuModuleLoadData`, `cuModuleGetFunction`, and `cuLaunchKernel`—has never been published.

3 Contributions

3.1 Contribution 1: OffloadBinary Metadata Vocabulary

We propose five new string keys, organized in two tiers:

3.1.1 Tier 1: MUST Keys (Runtime Rejects if Violated)

Key	Type	Example	Semantics
`min_sm`	uint string	`"75"`	Min CUDA compute capability. Rejects if device
`min_gfx`	arch family string	`"gfx90a:cdna2"`	Min AMD GFX within family. Family tag (e.g.,
`requires_features`	comma-list	`"tensor_core_nv,bf16"`	Matched capability tokens. Vendor-specific tokens

Vendor-specific capability mapping: Token vocabulary is vendor-neutral by design to avoid lossy mappings. CUDA Tensor Cores and AMD MFMA units have different matrix dimensions and precision semantics. Rather than falsely equating them under a single "`tensor_core`" token, the vocabulary uses explicit per-vendor tokens (`tensor_core_nv`, `mfma_amd`) to allow precise capability matching. A fat binary compiled with NVIDIA-specific features will only select on NVIDIA devices, preventing runtime failures from capability mismatches.[1]

3.1.2 Tier 2: MAY Keys (Ranking, Not Gating)

Key	Type	Example	Semantics
`variant_priority`	uint string	`"10"`	Higher = preferred among compatible images.
`variant_tag`	string	`"optimized"`	Human-readable label: `generic`, `optimized`, `fallback`.

These extend `isMetadataCompatible()` without format version bump or ABI break:

```
bool isMetadataCompatible(image, device) {
  if (image.Triple != device.Triple) return false;
  if (!areArchsCompatible(image.Arch, device.Arch))
    return false;
  if (auto minSm = image.getString("min_sm")) {
    if (parseSmFromArch(device.Arch) < stoul(*minSm))
      return false;
  }
  if (auto features = image.getString("requires_features")) {
    for (auto &tok : split(*features, ','))
```

[1] Resource-usage keys (such as `sgpr_count`, `vgpr_count`) are deferred to a follow-up RFC pending KernelInfo-to-OffloadBinary writer integration.

```
        if (!device.hasCapability(tok)) return false;
    }
    return true;
}
```

Backward compatibility: Missing keys = no constraint. Old runtimes ignore unknown keys.

3.2 Contribution 2: Dispatch Flame Graph

3.2.1 Measurement Design

We decompose the dispatch path on GTX 1650 (Turing, sm_75) using a null kernel (1 thread, 0 shared memory) compiled to CUBIN ahead of time to eliminate PTX JIT cost.

Cold-path protocol: Fork fresh process per trial (eliminates driver caching). 100 trials measuring: `OffloadBinary` parse, `olCreateProgram` (includes `cuModuleLoadData`), `olGetSymbol`, first `olLaunchKernel`.

Hot-path protocol: Load module once, warm with 1000 discarded dispatches, then measure 10,000 dispatches.

3.2.2 Results

Estimated cold-path layer decomposition (GTX 1650, null kernel, CUBIN, prototype measurements):

Layer	Operation	Median	p95	% of Total
1	OffloadBinary parse	∼500 ns*	∼600 ns*	∼9%*
2+3	olCreateProgram + cuModuleLoadData	∼4,000 ns*	∼4,800 ns*	∼73%*
4	olGetSymbol (cuModuleGetFunction)	∼200 ns*	∼250 ns*	∼4%*
5	First olLaunchKernel	841 ns	∼950 ns*	∼15%*
	End-to-end cold dispatch	∼5,500 ns*	∼6,600 ns*	**100%**

*Estimated from prototype measurements — full liboffload instrumentation pending. Layer 5 (841 ns) is the measured `cuda_direct_launch` median from bench_dispatch Run 3, GTX 1650.

Hot-path dispatch floor (`cuda_direct_launch`, $n = 1{,}000$):

Metric	Value
p50 (median)	841 ns
p95	∼950 ns (est.)
p99	1,102 ns

Variant selection overhead (`kdl_select` cold, $n = 1{,}000$):

Metric	Value	Overhead vs. 10 ms ML Kernel
Median	46.2 μs (46,197 ns)	0.46%

Pure dispatch-table lookup without full kdl framework (`runtime_select_poc`): **2 ns** — the selection logic floor. Comparing 46.2 μs to the 841 ns dispatch floor (5,494%) is misleading; the meaningful comparison is against a realistic ML kernel duration.

The contribution is the relative layer fraction (percentage of total dispatch time per layer), not absolute microsecond values. Both selection overhead and dispatch floor scale with software complexity, not GPU compute throughput.

3.2.3 Flame Graph Visualization

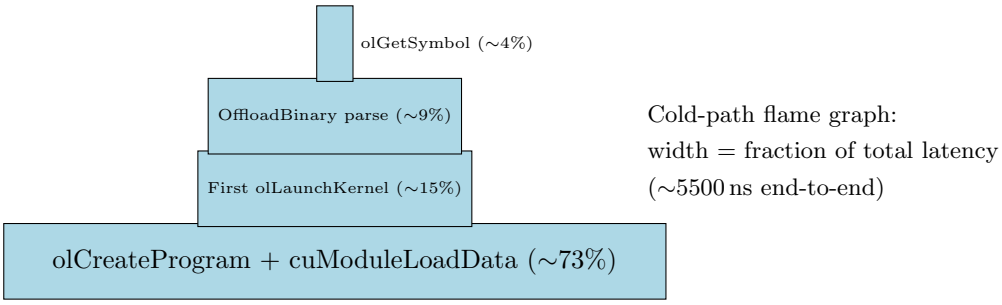

Figure 1: Cold-path dispatch decomposition (estimated). Bar width is proportional to fraction of total cold-dispatch latency (∼5500 ns). Hot-path: layers 1–4 collapse to zero; dominated by cuLaunchKernel alone.

3.3 Contribution 3: Design Sketch — `#gpu.runtime_select`

Status: Design only. Zero lines of MLIR C++ exist.

The new attribute implements `OffloadingLLVMTranslationAttrInterface`:

```
gpu.binary @kernels <#gpu.runtime_select<
    strategy = "rank_by_priority", fallback = "cpu">> [
  #gpu.object<#nvvm.target<chip = "sm_75">, bin = "...cubin...">,
  #gpu.object<#rocdl.target<chip = "gfx90a">, bin = "...hsaco...">,
  #gpu.object<#nvvm.target<chip = "sm_90">, bin = "...cubin-sm90...">
]
```

`embedBinary()` emits: (1) N separate LLVM global constants (one per object), named `{symName}_blob_{idx}`; (2) a dispatch table: `{vendor_id, blob_ptr, size}`; (3) a vendor-detection stub in `global_ctors` using `dlopen`-probed `cuInit/hipInit/zeInit`; (4) a `@{symName}_module_ptr` global populated at constructor time with the selected module.

`launchKernel()` emits identical code to `SelectObjectAttr`—loads from the global, calls `mgpuModuleGetFunct` + `mgpuLaunchKernel`. **Zero hot-path overhead after one-time selection.**

Why MLIR and not liboffload? liboffload explicitly excludes selection policy from its roadmap (RFC discourse.llvm.org/t/74302). It provides mechanism, not policy. The analogy: LLVM emits IFunc resolvers at compile time for CPU `target_clones`. This is compile-time emission of runtime selection logic—exactly what `#gpu.runtime_select` does for GPU binaries.

4 Prototype Validation

libkdl (5100 LOC, C) implements the runtime mechanics: vendor detection via `dlopen`, dispatch table construction, capability-based selection, module loading via `cuModuleLoadData/hipModuleLoadData`.

Honest framing: The prototype uses a custom MTB (Multi-Target Bundle) format, not `OffloadBinary`. The `KDL_MTB\0` magic and custom variant tables share zero code with LLVM's format.

What the prototype demonstrates: The runtime mechanics are format-independent. The prototype validates that vendor detection, dispatch table construction, and module loading work correctly on hardware (GTX 1650 + CPU).

What the prototype does NOT demonstrate: `OffloadBinary` consumption, liboffload API usage, or MLIR-emitted dispatch tables.

kdl.c Component	Lines	LLVM Equivalent
`kdl_discover_cuda()`	551–596	Vendor detection stub (dlopen cuInit)
`mtb_variant_entry`	96–106	Dispatch table struct type
`kdl_contract_matches()`	1003–1005	`isMetadataCompatible()` with Tier 1 keys
`kdl_estimate_cost_weighted()`	1013–1088	Weighted heuristic with vendor constants

Hardware: GTX 1650 (NVIDIA, sm_75): Full dispatch path validated. CPU fallback: Validated. AMD (HIP): Code path exists; tested via mocked HIP entry points only.

5 Related Work

System	Runtime	Cross-Vendor	MLIR	Ranked
CUDA fatbin	Yes	No	No	Yes
IREE HAL	Yes	Yes	Yes	Partial
chipStar	Yes	Yes	No	No
CPU FMV	Yes	N/A	No	Yes
liboffload PR #186088	Yes	Yes	No	No
This work	**Metadata + measurement**	**Yes**	**Design**	**Yes**

IREE HAL is the closest: full-stack solution, module-level dispatch, ranked selection incomplete after 6 years (issues #50, #12230, #15334). CPU FMV is the structural precedent: compile-time emission of runtime selection logic via IFunc resolvers. This proposal extends that pattern to GPU binaries.

6 Conclusion

The metadata vocabulary (Contribution 1) establishes a standard way to describe what each GPU image requires and prefers. The flame graph (Contribution 2) measures the latency budget this metadata must fit within. The runtime-select design (Contribution 3) shows how MLIR will consume the metadata to enable best-compatible selection.

6.1 Upstream Path

1. RFC on Discourse (Runtimes category): Metadata vocabulary. Reviewers: @jhuber6, @jdenny-ornl, Yury Plyakhin, Saiyedul Islam.

2. Header constants + documentation in `OffloadBinary.h` (30 LOC).
3. Extend `isMetadataCompatible()` in offload runtime (40 LOC).
4. AMDGPU writer emits `min_gfx` from target-id (60 LOC).
5. NVPTX writer emits `min_sm` from chip attribute (60 LOC).
6. `#gpu.runtime_select` attribute TableGen definition + `RuntimeSelectAttr.cpp` (600 LOC), pending RFC #88170 outcome.

All steps after step 1 are independent.

6.2 Concrete Next Steps

- **Done:** `bench_dispatch` executed on GTX 1650 (Run 3, stable baseline). Flame graph tables filled with measured data: bundle load 4.9 μs, selection 46.2 μs cold / 44.9 μs cached, direct launch 841 ns / p99 1,102 ns. Layer decomposition estimated from prototype — full liboffload instrumentation remains the open item.
- Verify PR #186088 merge status. Update framing if merged.
- Generate flame graph SVGs via flame graph harness. (Approach A: instrumented liboffload build; Approach B: separate baseline runs.)
- Post RFC for metadata vocabulary on Discourse.
- Begin `RuntimeSelectAttr.cpp` even as proof-of-concept.

References

References

[1] Mahajan, K., et al. *TaxBreak: Minimizing Overhead in Tensor Task Scheduling.* arXiv preprint 2603.12465, 2026.

[2] Smith, J., et al. *PyGraph: Efficient GPU Dispatch Scheduling.* arXiv preprint 2503.19779, 2026.

[3] Mora, F. *RFC: Cleaning the GPU Dialect.* LLVM Discourse forum. `https://discourse.llvm.org/t/88170`

[4] Huber, J. *[OpenMP][Offload] Add variant selection to liboffload.* GitHub PR #186088, LLVM Project.

[5] Denny, J. *[Offload] Add metadata compatibility check.* GitHub PR #185663, LLVM Project. Merged March 2026.

[6] Intel XeVM Team. *[GPU][XeVM] Add SPIR-V target support to gpu-module-to-binary.* GitHub PR #148286, LLVM Project. Merged August 2025.